

L3 - OOP

- Instead of keeping data & the funcⁿ separate we could combine them to form objects.

- Object has -

Attributes - data describing the object

Methods - operations that the object can perform.

Eg. Point → class
 x, y → attributes
 distance()
 translate() } methods

The object combines the state of an entity with the operations that act on that state.

- A class is a blueprint for creating the objects. An object is a particular instance of a class. The class defines what it has & what it can do.

∴, 1 class → many objects

Eg. class Point:

def __init__(self, x, y):

← initialiser

self.x = x

self.y = y

method

def distance_from_org(self):

return (self.x**2 + self.y**2)**0.5

P = Point(3, 4)

Q = Point(6, 8)

∴, 1 class → many independent objects

- A funcⁿ is defined independently and receives the data it needs as arguments.

- A method is funcⁿ defined in a class & is called on an obj. Eg. `P.distance()`. A method is associated with the object & can directly access that object's attributes through `self`.
- A special `--str--` method. Python automatically calls `--str--` when the object is used with `print()`.

Eg. `def --str--(self):`
 `return "(" + str(self.x) + "," +`
 `str(self.y) + ")"`

`print(P)` → `P.--str--()`
 ↓

tells Python how to represent objects of your class as a string.

- We can control the method for access & modification.

`get-y()` → reads

`set-y()` → modifies

↳ It helps the class to take the responsibility for maintaining the validity of its own state. It reduces the possibility of accidents.

- A class attribute belongs to the class rather than to a particular object.

class Student:

`university = "IIT"`

`def --init--(self, name):`

`self.name = name`

`s1 = Student("Alice")`

`s2 = Student("Bob")`

`s1.name = Alice, s2.name = Bob`

`s1.university = s2.university = IIT`

→ class attribute
 → object attribute

- We use a class attribute when a value is common to all the objects of the class.